

JVD Internal Management System

Product Improvement Spec · v1.0 FINAL · 2026-07-04

JVD Product Improvement Spec — Brand Colors, Notifications, Email, Documents, Workflows, Roles

Date: 2026-07-04 · Companion to `SYSTEM_AUDIT_AND_REFACTOR_PLAN.md` and `DESIGN_DIRECTION.md` Addresses six raised concerns, each grounded in the current code.

1. Brand Colors — using red / blue / yellow without an unappealing website

The problem: JVD's trademark triad (red, blue, yellow) at full strength is a primary-color scheme — it reads as loud and toy-like when spread across a UI. But you don't have to spread it. Professional brand-heavy apps follow a **90 / 9 / 1 rule**: ~90% of every screen is neutral (white, grays, near-black text), ~9% is ONE brand accent, ~1% is everything else. The logo keeps the full triad; the interface uses the triad *semantically*.

The trick that solves both your color problems at once: JVD's three brand colors map perfectly onto the three universal UI meanings. Instead of decorating with them, assign each one a job:

Brand color	Refined UI shade	Its ONLY job in the UI
JVD Blue	Deep/royal blue <code>#1D4ED8</code> (hover <code>#1E40AF</code> , tint <code>#EFF6FF</code>)	The interactive color. Links, active nav item, focused inputs, selected rows, progress bars, charts, "info" states. If it's blue, you can click it or it's telling you where you are.
JVD Red	<code>#DC2626</code> (tint <code>#FEE2E2</code>)	Danger only. Delete buttons, destructive confirmations, errors, overdue/rejected statuses. Never decoration. If it's red, be careful.
JVD Yellow	Amber <code>#D97706</code> on tint <code>#FEF3C7</code>	Warning/pending only. Pending-approval pills, expiring documents, low stock. Never decoration. If it's yellow, something is waiting.
(supporting, non-brand)	Green <code>#16A34A</code> on <code>#DCFCE7</code>	Success/approved/active — required for status language; not a brand color, used only in pills and confirmations.
Neutrals	<code>#FAFAFA</code> bg · <code>#FFFFFF</code> surface · <code>#E8E8E8</code> borders · <code>#171717</code> text · <code>#737373</code> muted	90% of every screen. Primary buttons stay near-black (per <code>DESIGN_DIRECTION</code>) so the brand colors keep their meaning.

Where the full triad IS welcome: the logo, the login screen (subtle — e.g. a thin tri-color top border or gradient accent line), loading screen, report/PDF letterheads, email headers. One tasteful tri-color accent line is a common device for triad brands — it says "JVD" without painting the app.

Result: the site looks neutral-professional, yet every colored pixel is a brand color doing a meaningful job. This IS the fix for concern #2 as well:

Color uniformity rules (enforced, not aspirational)

1. **Tokens only.** All colors come from the Tailwind theme tokens above. No raw hex / arbitrary `text-[#...]` classes in pages — add an ESLint rule to reject arbitrary color values.
 2. **Five statuses, one component.** Every status string in the system maps to exactly one of: `info` (blue), `warning` (amber), `danger` (red), `success` (green), `neutral` (gray) — rendered ONLY by the shared `StatusPill`. Create one mapping file (`constants/statusColors.ts`) for all modules; delete per-page color logic.
 3. **Blue = interactive** is a hard rule: nothing decorative may be blue, and no interactive element may be gray-styled.
 4. **One primary button per view**, near-black. Everything else is bordered/ghost.
 5. Charts use blue as the first series, then a fixed ordered palette (blue → slate → amber → green) — same order on every chart in the app.
-

2. Notifications — why they feel messy and unreliable, and the redesign

What the code does today (`app/Http/Services/NotificationService.php` , 10 static methods): - Every event fans out to **hardcoded role lists** — e.g. every PO submission notifies all of `super_admin`, `executive_vice_president`, `purchasing_manager`, `accounting_executive`. Executives receive *everything from every module*, which is why it feels like noise. - Sends are **synchronous** in the request; a mail failure can break or slow the user's action (part of "unreliable"). - No user preferences, no dedup, no grouping, no digest. Recipient logic is also duplicated inside controllers (`CashBudgetRequestController` builds its own recipient lists).

Redesign — notify the person whose turn it is, not the org chart:

1. **Event-driven core.** Each domain action fires a Laravel Event (`PurchaseOrderSubmitted` , `CashBudgetApproved` , ...). Queued listeners create notifications. Controllers stop knowing who gets notified.
 2. **Two classes of notification** with different treatment:
 3. **Actionable** ("your turn to approve X") → goes ONLY to users who can act on the current workflow step (see §5). Delivered in-app + push/email immediately. Disappears from the "needs action" list once someone acts — this alone removes most of the mess.
 4. **Informational** ("X was approved") → in-app only by default, to the requester + explicitly subscribed watchers, batched into digests for everyone else.
 5. **notification_preferences table** — user × category × channel (in_app / email / digest) with per-role sane defaults. A settings page renders it as a simple toggle grid.
 6. **Reliability:** all sends queued (`ShouldQueue`), retries with backoff, `failed_jobs` monitored (Phase 1's Sentry catches failures). Real-time delivery to the bell icon via Reverb private channel per user — replaces polling.
 7. **In-app UX** (DataTable-era bell panel): grouped by category with counts, actionable items pinned on top with inline Approve/View buttons, "mark all read", deep links to the exact record.
-

3. Email flooding — the digest engine

Today: ~42 direct mail/notify call sites; approval-chain emails dump full line items into inboxes; several people get an email for every step of every chain.

Fix — three delivery tiers, enforced centrally (no controller may call `Mail::to` directly; everything routes through one `Mailer` service):

Tier	What qualifies	Delivery
Immediate	It's your turn to act (approval requests with action links, via <code>PublicRequestActionController</code>); security (password, 2FA); customer-facing mail (portal links, contracts, booking confirmations)	Single email, queued, sent now
Digest	Everything informational: status changes, submissions you're watching, completions	One daily (configurable to hourly/weekly per user) summary email: grouped by module, one line per event, links into the app
In-app only	Low-value chatter (task assigned to someone else in your team, chat mentions when online)	Never emailed

Rules: emails contain a summary + a button into the app — never full line-item tables (the app is the source of truth; it also avoids leaking financial detail into inboxes). Every email footer links to the preferences page. Suppress duplicate sends for the same event+recipient within a window (dedup key on the queued job).

4. Company Documents — from scattered uploads to a real Document Repository (DMS)

Today: documents live in at least four unrelated tables (`ProcurementDocument` — which already has ad-hoc links to supplier/customer/job order/work order/trip ticket/driver/inventory — plus `PassportCaseDocument` , `JobApplicationDocument` , `LegalDocument`), and generated artifacts (invoices, contracts, payslips, PDFs) aren't filed anywhere at all. There's no single place to answer "show me every document about customer X" or "where is the signed hardcopy?"

Target: one `documents` table, everything else links to it.

```

documents
  id, doc_number (auto: JVD-DOC-2026-00001), title, category_id, tags[]
  storage_type: soft | hard | both
  file_path, mime, size, checksum          -- soft copy (S3/local private disk)
  physical_location, custodian_id          -- hard copy tracking ("Cabinet 3, Folder B", who holds
it)
  issue_date, expiry_date                  -- drives expiring-document alerts (visas,
accreditations, permits)
  status: draft | active | archived | disposed
  uploaded_by, source: uploaded | generated | portal
  retention_until

document_links (polymorphic many-to-many)
  document_id, linkable_type, linkable_id -- one document can attach to a Customer AND its
JobOrder AND its Invoice

document_versions
  document_id, version, file_path, uploaded_by, created_at -- re-uploads never overwrite

```

Key behaviors: 1. **Auto-filing**: when the system generates an invoice PDF, contract, payslip, or PO, an event listener registers it in `documents` with links to its records. The repository becomes complete without anyone doing filing work. 2. **Hardcopy tracking** is first-class: `storage_type=hard` documents have location + custodian and can be checked out/in (a simple `document_movements` log answers "who has the folder"). 3. **Expiry engine**: scheduled job scans `expiry_date`, feeds the notification digests ("3 supplier accreditations expire this month"). 4. **Access control**: category × role via the \$6 permission system; documents inherit module scoping from what they link to. 5. **Migration**: keep old tables read-only, backfill them into `documents` with links, point modules at the repository, then retire them. Existing upload-validation hardening (MIME whitelist, private disk) carries over as the single upload path. 6. **Frontend**: one **Documents** module using the DataTable pattern (filters: category, module, storage type, expiry, tags; preview drawer; version history), plus a "Documents" tab on every entity page showing its linked docs.

5. Approval chains — replace per-module status spaghetti with one workflow engine

Today: each module hand-rolls its chain as status strings inside controllers — e.g.

```

CashBudgetRequestController manually walks draft → pending_accounting → pending_super_admin →
approved → disbursed with inline if ($previousStatus === ...) checks, its own emails, and its own
approver capture ( approved_by , super_admin_approved_by columns). POs and WOs each re-implement the
same idea differently. Adding a step or changing an approver means editing controller code. There's no
unified history and no "everything waiting on me" view.

```

Target: one data-driven engine, chains defined as data:

```

workflow_definitions  (id, module, name, active)
workflow_steps        (definition_id, order, name,
                      approver_type: permission | role | user,
                      approver_value,                -- e.g. 'cash_budgets:approve_step2'
                      condition_json)                -- e.g. {"amount_gt": 50000} → step
only applies above threshold
workflow_instances    (definition_id, subject_type, subject_id, current_step, status)
workflow_actions      (instance_id, step, user_id, decision: approved|rejected|returned, comment,
                      acted_at)

```

One `WorkflowService` (`submit` / `approve` / `reject` / `return` / `whoCanActNow`) replaces every controller's status logic. What this buys:

1. **Configurable without code** — an admin screen (super_admin only) edits steps, approvers, and amount thresholds per module. This is exactly the client's "some roles have added or customized processes" requirement.
2. **Conditional routing** — small cash budgets skip the super-admin step; big POs add one. Today that's impossible without new code.
3. **Full audit trail** — `workflow_actions` is the immutable who/when/what-comment history for every approval, feeding the existing AuditLogger.
4. **One "Approvals" inbox** — `whoCanActNow` powers a single page listing everything awaiting *you* across all modules (the existing `dashboards/approvals` endpoint becomes this), with Approve/Reject inline. Pairs with §2: actionable notifications are generated from step transitions, so exactly the right people are pinged, once.
5. **Visualization** — the existing `PipelineVisualizer` component renders the instance's steps (done ✓ / current ● / upcoming ○) on each record's page, using §1 semantics: amber = waiting, green = done, red = rejected.
6. **Email approvals keep working** — `PublicRequestActionController` 's signed action links plug into `WorkflowService::approve` instead of module-specific handlers.

Migration order: Cash Budget first (worst offender, clear 3-step chain), then PO, then WO; each migration deletes that controller's status spaghetti. Old status columns are kept in sync during transition, then dropped.

6. Roles — keep the simplicity, add the flexibility clients want

Today: two half-systems. (a) Hardcoded role names scattered through `routes/api.php`, controllers, and even `NotificationService` (`whereIn('role', ['super_admin', ...])` in 10+ places) — inflexible. (b) The `role_permissions` table — flexible but only expresses view/create/edit/delete per module, which can't say "this role approves step 2 of cash budgets" or "this role can disburse but not approve."

Target: named abilities + roles as data.

1. **Ability registry.** Replace the fixed CRUD grid with named abilities per module, kept in one registry file so the admin UI and middleware share it: `cash_budgets: view, create, edit, delete, approve_accounting, disburse`, `documents: view, upload, manage_hardcopy`, `invoices: finalize`, ... CRUD stays as the default four; modules add verbs as needed. (Schema: replace the four `can_*` booleans with an `abilities` JSON array or a pivot — one migration, `getAllPermissions()` keeps its shape with extra keys so existing checks don't break.)
2. **Roles become rows, not code.** A `roles` table (name, label, is_system) with the current hardcoded roles seeded as system roles. Admins can create custom roles ("Senior Travel Agent") by cloning a template role and toggling abilities — no deploy. `CheckRole` middleware keeps its hybrid form; the dynamic side now checks abilities.
3. **Everything references abilities, not role names.** The workflow engine (§5) targets `approver_type: permission`; `NotificationService`'s role lists become "users with ability X"; new routes use `module:ability` middleware. The hardcoded role lists get burned down module-by-module during Phase 2's controller refactor (they're being rewritten anyway).
4. **Per-user overrides** (grant/revoke single abilities for one person) — small table, checked after role abilities; covers the "one person temporarily covers approvals" reality without inventing roles.
5. **Guardrails:** `super_admin` bypass stays; ability changes are audit-logged; the role editor UI shows *effective* permissions (role + overrides) so admins can see what a person can actually do.

This keeps the current model as a subset (nothing breaks on day one) and delivers the client ask: custom roles, custom processes (via §5 workflow steps targeting abilities), custom permissions — all admin-editable.

7. Role dashboards — stop hand-building N dashboards, build widgets once

Today: 9 hardcoded dashboard pages (`pages/dashboards/`) backed by a 713-line `DashboardController` with one bespoke method per role. Every new role or tweak means editing both — which is why "we can't fix and implement it."

Target: dashboards are configuration, not code.

1. **Widget registry.** Build ~12–15 small, self-contained widgets, each = one focused API endpoint + one frontend component with a fixed data contract: *Awaiting my approval* (from the §5 workflow engine), *Revenue this month*, *Collections due/overdue*, *Expiring documents* (from §4), *Fleet availability today*, *My tasks*, *Recent activity*, *Bookings this week*, *Pending payroll*, *Low stock*, *New applications*, *Open work orders*.
2. **Role → default layout** mapping stored as data (JSON layout per role, admin-editable). A new role gets a dashboard by picking widgets, not by writing a page.
3. **Per-user customization** (optional, later): show/hide/reorder, saved to the user's profile.
4. **Three archetypes cover everyone** — build these defaults first:
5. **Executive** (`super_admin`, EVP): money overview + approvals inbox + alerts.
6. **Module manager** (accounting, procurement, operations, HR, logistics): their module's KPIs + work queues.
7. **Operator** (agent, driver, mechanic): "my work today" — tasks, assigned trips, deadlines. Nothing org-wide.

8. Each widget endpoint is small, cached (60s Redis), and permission-checked by ability — a widget simply doesn't render if the user lacks its ability. `DashboardController` shrinks to a layout resolver + the widget endpoints.
-

8. Sales UI & the "what fields do we even need?" problem — model the catalog, not the screens

The checkout/transactions UI feels AI-generated because it was built screen-first: fields were guessed, not derived from how JVD actually sells. The fix is a **service catalog**: define each thing you sell as data, and let the UI render from that definition. (This also collapses the 2,685-line `FixedPackages.tsx` — packages become catalog rows, not code.)

8.1 The domain workshop (the anti-vibe-coding process — do this before coding)

One 2-hour session per category with the people who actually sell it. For each sellable category, fill one worksheet:

1. What does the customer ask for? (their words)
2. What do YOU need to know to quote a price? ← these are the **required input fields**
3. What changes the price? ← the **pricing model**
4. What must appear on the invoice / contract / receipt? ← the **display fields**
5. What can go wrong / what do you double-check? ← **validations** (e.g. bus already booked, pax > seats)
6. Get their current paper form / old quotation — it's the field list, already validated by years of use.

8.2 Starter field matrices (to validate in the workshop, not to skip it)

Bus rental: pickup location + destination(s) · departure & return date/time (multi-day?) · pax count → validate against bus `custom_seats` · bus type/specific unit · with driver? · route notes/estimated km · overnight driver allowance? · client type (school/company/individual) · deposit % · price = per-day or per-trip + distance factor.

Educational tour: school/organization + contact person · destination(s) · tour date(s) · students + teachers/chaperones count (chaperone free ratio?) · grade level · itinerary (reuse `Itinerary` model) · inclusions (meals, entrance fees, insurance, certificates) · per-head price w/ minimum pax · permit/compliance reference · payment schedule (downpayment + balance before tour date — reuse `PaymentSchedule`).

Same exercise for: international package, visa processing, hotel/flight booking, custom transaction.

8.3 Implementation

```
service_categories: id, name, pricing_model (per_pax|per_day|flat|per_head_min_pax),
                    field_schema JSON      -- [{key, label, type, required, options,
validation}]
services:           category_id, base rates -- (existing table, gains category link)
```

- Checkout reads `field_schema` and renders **only that category's fields** — a dynamic form (react-hook-form + zod generated from the schema). Adding a category or field = admin data change, not a deploy.
- Captured answers save to `custom_transaction_detail` (exists) — with every field listed in validation rules (the known `validate()` nested-key gotcha).
- The checkout becomes a 5-step wizard (per `DESIGN_DIRECTION.md` §2.1): **category** → **details (schema-driven)** → **passengers** → **pricing review** → **payment/contract** — each step small, nothing overflows.

9. Data integrity — why data gets switched, overwritten, and forgotten (and the six rules that stop it)

Measured evidence in the current code: - `BillingController` performs invoice creation and multi-step updates with `zero DB::transaction` (only 10 transaction call sites exist in all 47 controllers). If anything fails mid-flow, earlier writes stay and later ones never happen → data "forgotten." - Availability/conflict checks (`InvoiceFinalizationService::finalizeWithinTransaction` bus/driver checks) are read-then-write **without row locks** — two simultaneous checkouts both pass the check → bookings "switched"/double-booked. - `$invoice->update()` is called repeatedly at different flow stages (lines ~379, 477, 584 of `BillingController`) — later stages can silently **overwrite** fields earlier stages set. - **Billing and Collections never post to `LedgerService / JournalEntry`** — there is no shared financial record between sales and accounting; each module's screens re-derive numbers, so they drift. - Laravel's `validate()` silently drops unlisted nested keys (documented gotcha) — fields the frontend sent arrive as `null` → "forgotten."

The six rules (this is "the right process" — memorize these, they're what separates professional backends from vibe-coded ones):

1. **One writer per field.** Every field has exactly one module that may write it; everyone else reads. Cross-module handoffs pass **IDs, never copied values**. If accounting needs the invoice total, it reads the invoice — there is no second copy to get out of sync.
2. **Every multi-write is a transaction.** If a request writes more than one row, wrap it in `DB::transaction` — all of it happens or none of it does. (Phase 2 rule: no controller/service may do sequential writes outside one.)
3. **Finalized records are immutable — snapshot at finalization.** When an invoice/PO is finalized, freeze the financial facts (line items, prices, customer details *as sold*) into its own rows. Later changes to services, prices, or customer records must never rewrite history. Corrections are **new records** (credit note, amendment — the `ContractAmendment` model already follows this pattern) — never edits to a finalized one.
4. **Money moves are append-only ledger entries.** Every financial event — invoice finalized (AR + revenue), payment received, PO liability, disbursement — posts a `JournalEntry` via the existing (currently unused-by-sales) `LedgerService`. Accounting reports read the ledger, not module screens. An invoice's payment status becomes **derived** (sum of its ledger payments vs total), not a hand-set field — a derived value can't be overwritten wrongly.
5. **Lock what you check.** Availability checks (bus, driver, stock) use `lockForUpdate()` *inside* the same transaction as the write, with DB unique constraints as the backstop. Check-then-write without a lock is a race condition, full stop.

6. **Statuses only move through the state machine.** No `->update(['status' => ...])` scattered in controllers — the \$5 workflow engine's guarded transitions are the only way a status changes, and every change is recorded in `workflow_actions`.

Supporting practices: FormRequests everywhere (kills the `validate()` nested-key loss — every accepted field is explicit); idempotency keys on payment webhooks (PayMongo retries must not double-post); DB foreign keys with `restrict` on finalized records; the AuditLogger already in place captures who changed what.

The corrected pipelines:

- **Sales → Accounting:** Checkout → `Invoice (draft)` → **finalize** [one transaction: snapshot line items → lock & create travel bookings → post AR journal entry] → payments recorded in Collections post their own ledger entries (never touching invoice rows) → payment status derived from ledger sums → accounting dashboards/reports read the ledger. `InvoiceFinalizationService` already has the right transaction-boundary skeleton (`finalizeWithinTransaction` / `afterCommit`) — the work is moving ALL invoice writes inside it, adding locks, and adding the ledger posting.
- **Procurement → Accounting:** PO approved (workflow engine) → snapshot line items → goods received → AP journal entry → disbursement posts payment entry → accounting reads ledger. The PO stops being edited after approval; changes = revision records.

Migration order: wrap existing money paths in transactions first (days, huge risk reduction) → derive payment status from payments → add ledger postings → add locks → then snapshots. Each step is independently shippable.

10. Where this fits the master plan

This spec	Lands in	Depends on
\$1 color system + uniformity rules	Phase 3 (tokens are the first Phase 3 task — <code>DESIGN_DIRECTION.md</code> \$1.1 updated to match)	—
\$2 notifications + \$3 email digests	Phase 2	Queues (Phase 2 item 1)
\$4 document repository	Phase 2–3	Queues, DataTable component
\$5 workflow engine	Phase 2	\$6 abilities
\$6 abilities/roles	Phase 2, before \$5	—
\$7 dashboard widgets	Phase 3 (frontend) + Phase 2 (endpoints)	\$5 (approvals widget), \$6 (ability gating)
\$8 service catalog + checkout wizard	Phase 3, workshop first (can start now — it's meetings, not code)	Design system components
\$9 data integrity	Phase 2, FIRST — transactions on money paths are arguably Phase 0-level urgent	—

Final Phase 2 build order: **transactions on money paths (\$9.1–2) → queues → abilities (\$6) → workflow engine (\$5) → ledger integration + derived statuses (\$9.4) → notifications/email (\$2–3) → document repository (\$4) → dashboard endpoints (\$7).**

See `IMPLEMENTATION_ROADMAP.md` for the consolidated, checklist-form plan across all documents.